

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
Навчально-науковий Фізико-технічний інститут
Кафедра інформаційної безпеки

ПРОТОКОЛ

до комп'ютерного практикуму

з дисципліни «Основи аналізу алгоритмів»

Тема: «Реалізація синтаксичного аналізатора та інтерпретатора команд СКБД
(Етап 1)»

Варіант № 2

Виконали:

студентка гр. ФБ-43 Цвілій Влада

студентка гр. ФБ-44 Коломоєць Ольга

Київ — 2026

Мета роботи: Ознайомлення із базовими принципами побудови систем керування базами даних (СКБД) та механізмами синтаксичного аналізу. Отримання практичних навичок проектування лексичного і синтаксичного аналізатора для SQL-подібної мови запитів, побудови абстрактного синтаксичного дерева (AST), створення інтерактивної консольної оболонки (REPL) із підтримкою багаторядкового вводу та надійної валідації синтаксису.

Завдання варіанту № 2

Реляційна модель із підтримкою агрегації даних (рядковий тип). Для спрощення реалізації всі стовпці утворюваних таблиць мають рядковий тип даних. Необхідно реалізувати синтаксичний розбір наступних команд:

1. `CREATE table_name (column_name *[INDEXED] [, ...]);` — створення таблиці з можливістю позначення індексованих стовпців.
2. `INSERT [INTO] table_name ("value" [, ...]);` — додавання рядка даних (значення обов'язково беруться у подвійні лапки `"`).
3. `SELECT [agg_function(agg_column) [, ...]] FROM table_name [WHERE condition] [GROUP_BY column_name [, ...]];`
 - Якщо `GROUP_BY` відсутній: після `SELECT` не вказуються стовпці (вибираються всі стовпці таблиці в порядку створення).
 - Якщо `GROUP_BY` присутній: після `SELECT` вказуються агрегатні функції `COUNT`, `MAX`, `LONGEST`.
 - Умова `WHERE` підтримує виключно бінарне порівняння менше (`<`) між стовпцем і константою або двома стовпцями.
4. Додаткове завдання (*): підтримка синтаксису індексації стовпців таблиці (ключове слово `INDEXED`) для створення структур швидкого пошуку на наступному етапі.

Опис інтерфейсу та архітектури програми

Програма розроблена мовою Python 3 і має модульну структуру:

- `main.py` — забезпечує інтерфейс командного рядка (REPL — Read-Eval-Print Loop). Надає зручні підказки для користувача (`>` для початку нової команди та `'...` для продовження багаторядкового вводу). Реалізує буферизацію тексту та поділ запитів за символом крапки з комою (`;`). Скінчений автомат у функції `split_by_semicolon` гарантує, що крапка з комою всередині подвійних лапок не розглядається як кінець команди.
- `parser.py` — модуль синтаксичного аналізу на базі регулярних виразів. Перевіряє

правильність запитів, валідує ідентифікатори за регулярним виразом `[a-zA-Z][a-zA-Z0-9_]*`, формує структуру синтаксичного дерева (AST у форматі словника Python) та перехоплює помилкові ситуації з інформативними повідомленнями.

Синтаксичний розбір та оцінка складності парсера

Згідно з методичними рекомендаціями, граматика запитів варіанту № 2 є регулярною і не містить вкладених підвиразів довільної глибини. Тому синтаксичний аналіз реалізовано за допомогою регулярних виразів (модуль `re`), що забезпечує швидкість виконання без зайвих бібліотечних залежностей.

Аналіз часової та просторової складності парсера:

- Часова складність: $O(L)$, де L — довжина введеного тексту команди в символах. Регулярні вирази скомпільовані без випадків катастрофічного повернення (*catastrophic backtracking*), оскільки розділювачі (дужки, коми, лапки) є детермінованими.
- Просторова складність: $O(L)$, необхідна для зберігання вхідного рядка та результуючого дерева розбору (AST).

Результати тестування парсера

Нижче наведено результати практичної перевірки роботи розробленої програми для всіх категорій команд та обробки виняткових ситуацій.

Тестування команди CREATE

Створення таблиці з підтримкою багаторядкового вводу та індексами (INDEXED)

```
> CREATE cats (  
... id INDEXED,  
... name  
... INDEXED  
... , favourite_food  
... ); // mewmewmewmew  
Parsed query structure:  
{  
  "type": "CREATE",  
  "table_name": "cats",  
  "columns": [  
    {  
      "name": "id",  
      "indexed": true  
    },  
    {  
      "name": "name",  
      "indexed": true  
    },  
    {  
      "name": "favourite_food",  
      "indexed": false  
    }  
  ]  
}  
... █
```

Тестування команди INSERT

Додавання рядка значень з обов'язковими подвійними лапками:

```
> insert into cats ("1", "Murzik", "Sausages");
Parsed query structure:
{
  "type": "INSERT",
  "table_name": "cats",
  "values": [
    "1",
    "Murzik",
    "Sausages"
  ]
}
```

Тестування команди SELECT

А) Базова вибірка без умов (відповідно до Варіанту 2 — після SELECT нічого не вказується):

```
> SELECT FROM cats;
Parsed query structure:
{
  "type": "SELECT",
  "table_name": "cats",
  "aggregations": [],
  "where": null,
  "group_by": null
}
```

Б) Фільтрація з літералом у лапках (WHERE name < "Murzik"):

```
> select from cats where name < "Murzik";
Parsed query structure:
{
  "type": "SELECT",
  "table_name": "cats",
  "aggregations": [],
  "where": {
    "left": "name",
    "operator": "<",
    "right": "Murzik",
    "is_literal": true
  },
  "group_by": null
}
```

В) Групування з агрегатними функціями (COUNT, MAX, LONGEST):

```
> SELECT COUNT(id), MAX(favourite_food), LONGEST(name)
... FROM cats
... GROUP_BY name, id;
Parsed query structure:
{
  "type": "SELECT",
  "table_name": "cats",
  "aggregations": [
    {
      "function": "COUNT",
      "column": "id"
    },
    {
      "function": "MAX",
      "column": "favourite_food"
    },
    {
      "function": "LONGEST",
      "column": "name"
    }
  ],
  "where": null,
  "group_by": [
    "name",
    "id"
  ]
}
```

Тестування обробки помилкових ситуацій

Парсер виявляє порушення граматики та логічні невідповідності:

1. Спроба вказати стовпці у SELECT без GROUP_BY:

```
> SELECT name, id FROM cats;
Error: Без GROUP_BY вираз після SELECT повинен бути порожнім (використовуйте: 'SELECT FROM ...')
```

2. Непідтримуваний оператор у WHERE (дозволено лише <):

```
> SELECT FROM cats WHERE name > "Murzik";
Error: Некоректна умова WHERE (очікується col < val або col < col2): 'name > "Murzik"'
```

3. Невідома агрегатна функція:

```
> SELECT AVG(id) FROM cats GROUP_BY name;
Error: Невідома або некоректна агрегатна функція: 'AVG(id)'
```

4. Значення в INSERT без подвійних лапок:

```
> INSERT INTO cats (123, "Murzik");  
Error: Некоректний синтаксис списку значень біля: '123, "Murzik"'
```

5. Дублювання імені стовпця при CREATE:

```
> CREATE cats (name, age, name);  
Error: Дублювання імені стовпця: 'name'
```

Висновки

Під час виконання 1-го етапу комп'ютерного практикуму було розроблено повноцінний консольний інтерпретатор та синтаксичний аналізатор (парсер) для реляційної системи баз даних за Варіантом № 2. Програма успішно обробляє команди CREATE, INSERT, SELECT із дотриманням усіх обмежень варіанта (підтримка багаторядкового вводу, символ закінчення команди крапка з комою, регістронезалежність ключових слів, відсутність стовпців у SELECT без групування, обов'язкові подвійні лапки в літералах, агрегатні функції COUNT, MAX, LONGEST). Також реалізовано синтаксичну підтримку додаткового завдання (*) — розпізнавання модифікатора INDEXED для індексації стовпців, що підготувало структури AST до інтеграції з модулем зберігання даних (database.py) на 2-му етапі.